

中山大學
SUN YAT-SEN UNIVERSITY

Assignment 1 实验报告

MDP

姓名　　：钟泓瑨

学号　　：23336338

教学班号　：202610271

专业　　：计算机科学与技术

院系　　：中山大学

2026 年 10 月 13 日

目录

1 实验要求及问题定义 **2**

2 MDP 建模 **2**

3 算法设计 **3**

4 实验结果 **5**

4.1 主实验结果 5

4.2 折扣因子对求解的影响 6

4.3 其他扩展测例 7

5 总结 **9**

1 实验要求及问题定义

- 将给定迷宫建模为马尔可夫决策过程（MDP），使用 Policy Iteration 或 Value Iteration 求解；本实验采用 Value Iteration。
- 状态：用智能体所在格子的坐标 (r, c) 表示，墙体不属于状态空间；`S` 和 `G` 分别为起点和终点。
- 动作：向北、东、南、西（N、E、S、W）四个方向移动，每次一格。
- 状态转移：移动结果确定；若目标位置为墙体或超出边界，则留在原地。
- 奖励与终止：每步奖励为 -1，包括撞墙和进入终点；到达终点后结束，终点价值为 0。
- 目标：在折扣因子 $0 < \gamma < 1$ 下，求使累计折扣回报最大的策略，即以最少步数从起点到达终点。

2 MDP 建模

将迷宫问题建模为马尔可夫决策过程 $\mathcal{M} = (\mathcal{S}, \mathcal{A}, P, R, \gamma)$，五个分量依次为状态空间、动作空间、状态转移概率、奖励函数和折扣因子。

状态空间 $\mathcal{S}$ 由所有可通行格子的坐标 (r, c) 构成，包含起点和终点，终点记为 s_g。行、列编号从 0 开始，分别向下、向右增大。

动作以行、列位移表示，上、右、下、左四个方向对应的动作空间为

$$\mathcal{A} = \{(-1, 0), (0, 1), (1, 0), (0, -1)\}.$$

用 $s + a$ 表示坐标与位移按分量相加的结果。若目标格可通行，则移至该格；撞墙或越界时留在原地。终点为吸收状态，转移函数为

$$f(s, a) = \begin{cases} s_g, & s = s_g, \\ s + a, & s \neq s_g \text{ 且 } s + a \in \mathcal{S}, \\ s, & \text{其他情况.} \end{cases}$$

对应的转移概率为

$$P(s' \mid s, a) = \begin{cases} 1, & s' = f(s, a), \\ 0, & \text{其他情况.} \end{cases}$$

非终点状态的每次动作奖励为 -1，包括撞墙、越界和进入终点。到达终点后回合结束，后续奖励为 0。奖励函数及终点价值为

$$R(s,a)=\begin{cases}-1, & s\neq s_g,\\ 0, & s=s_g,\end{cases}\qquad V(s_g)=0.$$

策略 π 根据当前状态选择动作，其价值函数定义为从状态 s 出发的期望累计折扣回报：

$$V^{\pi}(s)=\mathbb{E}_{\pi}\left[\sum_{t=0}^{\infty}\gamma^{t}R(s_t,a_t)\,\middle|\,s_0=s\right].$$

求解目标是得到最优价值函数 $V^{*}(s)=\max_{\pi}V^{\pi}(s)$ 及对应策略 π^{*}。对于终点可达的状态，若经过 d 步到达终点，回报为 $-\sum_{t=0}^{d-1}\gamma^{t}$。每多走一步，回报减少 γ^{d}，因此最大化回报等价于选择到终点的最短路径。

3　算法设计

价值迭代的关键是反复使用 Bellman 最优方程更新每个状态的价值。在第 k 轮迭代中，代码中的 `values` 表示 V_k。对于状态 s 下的动作 a，`maze.step` 返回执行该动作后的状态 s'、即时奖励 r 和终止标志，`action_value` 随后计算

$$Q_k(s,a)=\begin{cases}r+\gamma V_k(s'), & \text{该动作后回合未结束},\\ r, & \text{该动作后回合结束}.\end{cases}$$

对应代码中的 `reward + (0.0 if done else gamma * values[successor])`。接着，`updated` 中对所有动作价值取最大值，实现了

$$V_{k+1}(s)=\max_{a\in\mathcal{A}}Q_k(s,a).$$

只有当所有状态的新价值都计算完毕后，`updated` 才会赋给 `values`，所以同一轮中的全部状态都依据同一个 V_k 更新，即同步价值迭代。相邻两轮的价值函数基本不再变化后，代码再次使用相同的动作价值计算，并选择使其最大的动作，从而实现 $\pi^{*}(s)=\arg\max_{a}Q^{*}(s,a)$。相关代码如下。

代码 1: Bellman 最优更新与策略提取

```
    def action_value(state: State, move: Move) -> float:
        successor, reward, done = maze.step(state, move)
        return reward + (0.0 if done else gamma * values[successor])
```

```
    while True:
        updated = {
            state: max(action_value(state, move) for move in MOVES)
            for state in maze.states
        }
        deltas.append(max(abs(updated[state] - values[state]) for state in
    maze.states))
        values = updated
        if deltas[-1] < tolerance:
            break

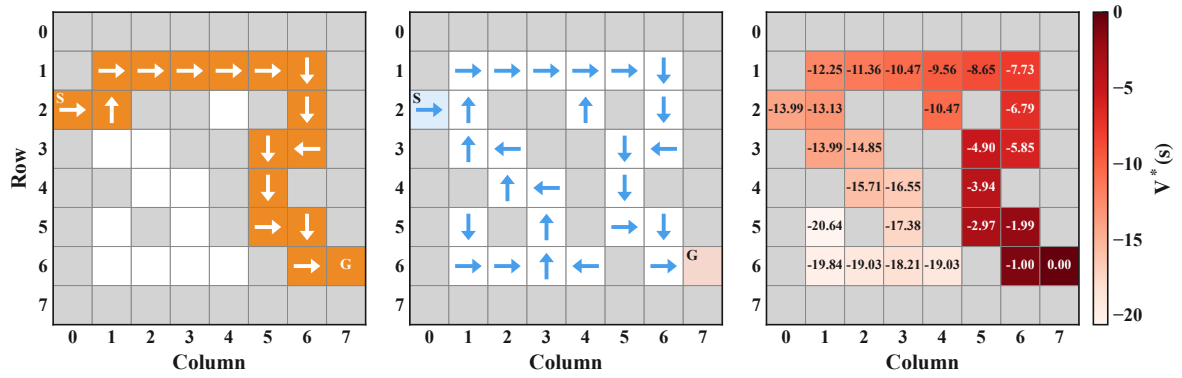
    policy = {
        state: max(MOVES, key=lambda move: action_value(state, move))
        for state in maze.states
        if state != maze.goal
    }
```

4　实验结果

除折扣因子的对比实验外，以下实验均使用 $\gamma = 0.99$、停止阈值 10^{-8}，价值初始值为 0。每张迷宫横图从左到右依次展示路径、策略和状态价值：橙色格子表示经过的路径，白色箭头表示行进方向，蓝色箭头表示各位置选择的动作，右图中的数值表示对应状态的价值。路径长度另用广度优先搜索进行核对。

4.1　主实验结果

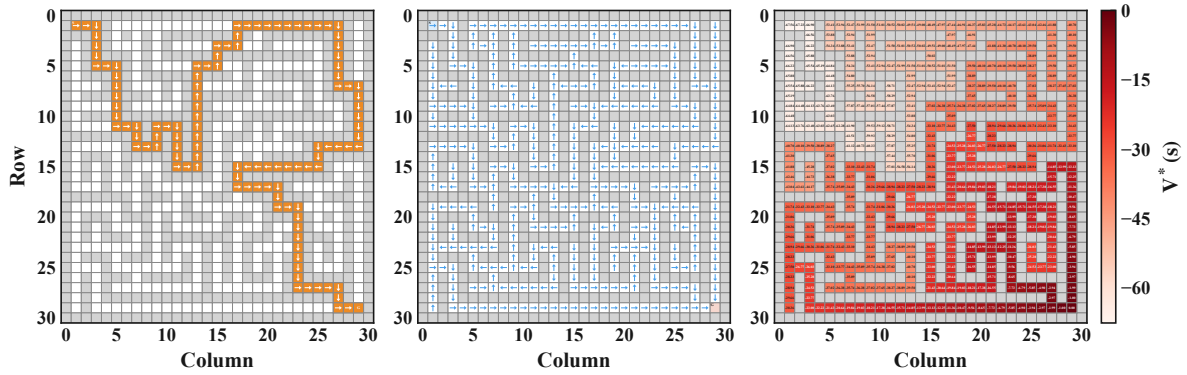
作业给定的迷宫为 8×8，共有 27 个可通行状态。价值迭代在 24 轮后停止，得到的路径从起点出发，绕过中间的障碍后到达终点，共 15 步，与广度优先搜索得到的最短路径长度一致。由于每步奖励为 -1，这条路径的未折扣回报为 -15，起点价值为 -13.994165。



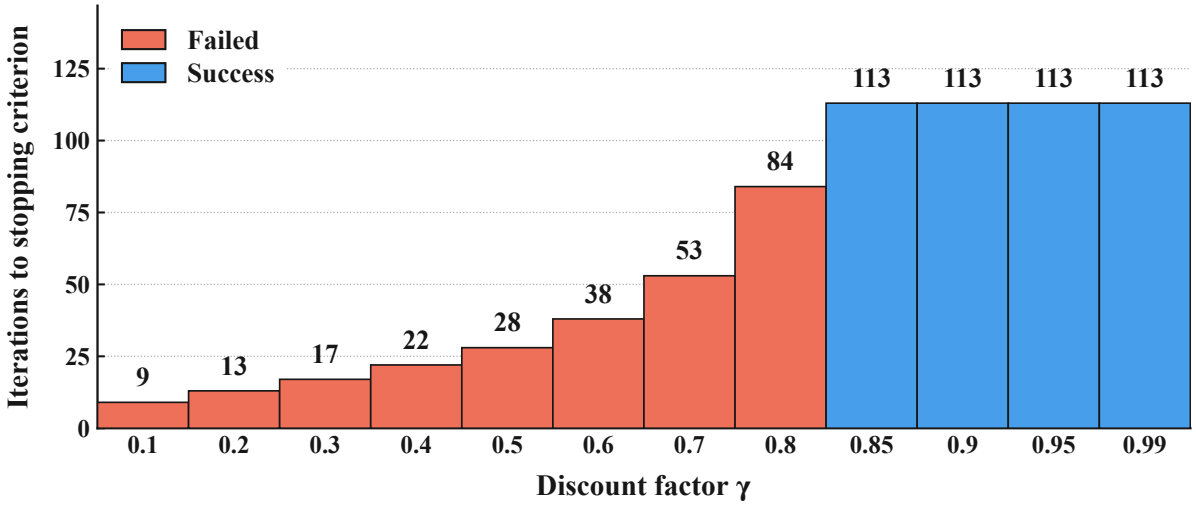
策略图不仅给出了起点到终点的行进方向，也给出了路径之外各个位置应采取的动作。价值图中，沿最短路线越接近终点，价值越接近 0。这里的 24 轮迭代多于路径的 15 步，是因为算法每轮更新整张价值表；该迷宫中距离终点最远的状态需要 23 步才能到达终点，直到第 24 轮检查时，所有状态的价值才都不再变化。

4.2　折扣因子对求解的影响

为了观察复杂路径下的求解结果，我另外定义了 `complex_large` 测例，将网格扩大到 31×31，包含 465 个可通行状态，并加入多处分支、死胡同和回路。起终点的曼哈顿距离为 56，但墙壁使最短可行路径达到 112 步。算法经过 113 轮更新后得到这条最短路径，起点价值为 -67.555447。下图依次展示路径、策略和价值分布。



固定其余条件后，我比较了从 0.1 到 0.99 的 12 个折扣因子。下图的柱高表示满足停止条件所需的迭代轮数；红色 Failed 表示所得策略无法到达终点，蓝色 Success 表示能够到达终点。



本次测试中，$\gamma \le 0.8$ 的各组均未能到达终点，其中 $\gamma = 0.1$、0.5、0.8 分别在第 9、28、84 轮停止。较小的 γ 使远期奖励的影响随 γ^t 迅速衰减，每轮价值变化很快低于停止阈值 10^{-8}，而终点信息此时尚未传播到起点。起点四个动作的估计价值仍然相同，程序按枚举顺序选择向上移动，撞墙后留在原地，之后又重复同一动作。因此，这些失败来自长路径上的折扣衰减与固定停止阈值共同造成的提前停止。

当 γ 为 0.85、0.9、0.95 和 0.99 时，程序均经过 113 轮更新得到 112 步的最短路径。作为对照，仅将 $\gamma = 0.8$ 的停止阈值收紧到 10^{-12}，程序也能得到同一长度的最短路径，验证了停止阈值对该组结果的影响。当 γ 更小时，动作价值之间的差异还可能被浮点舍入抹去，因此仅收紧停止阈值也未必足够。

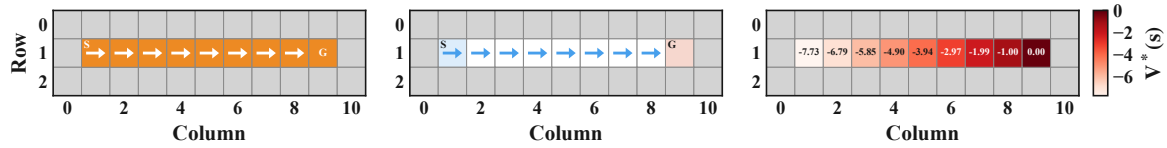
4.3　其他扩展测例

我还设计了以下测例，分别检查不同迷宫结构对结果的影响。它们的路径长度都与广度优先搜索一致，汇总如下；随后给出每个测例的设计目的和三图对照。

测例	网格大小	状态数	路径步数	迭代轮数
`corridor`	3×11	9	8	9
`open_room`	6×9	28	9	10
`detour`	7×9	31	14	15
`dead_ends`	9×13	47	16	21
`loop`	7×9	20	10	11
`larger`	15×15	102	40	47

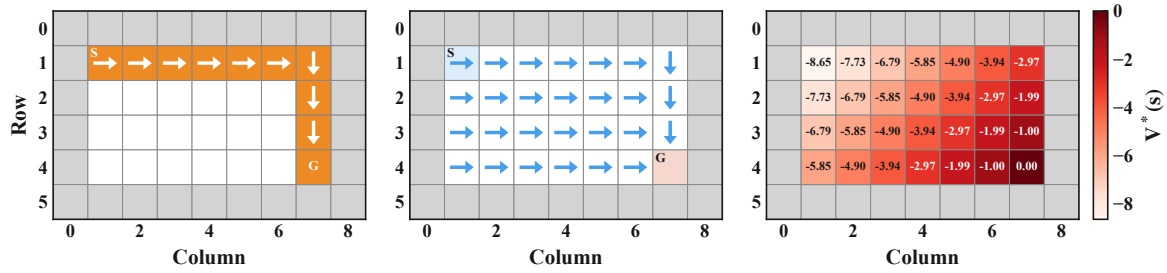
(1) 直线走廊：`corridor`

这个测例没有岔路，用来观察价值能否沿通道逐步传播。策略始终朝终点方向移动，得到长度为 8 的路径，9 轮后停止。各位置的价值随剩余步数变化，起点价值为 -7.725531，与沿这条走廊累计得到的折扣回报一致。



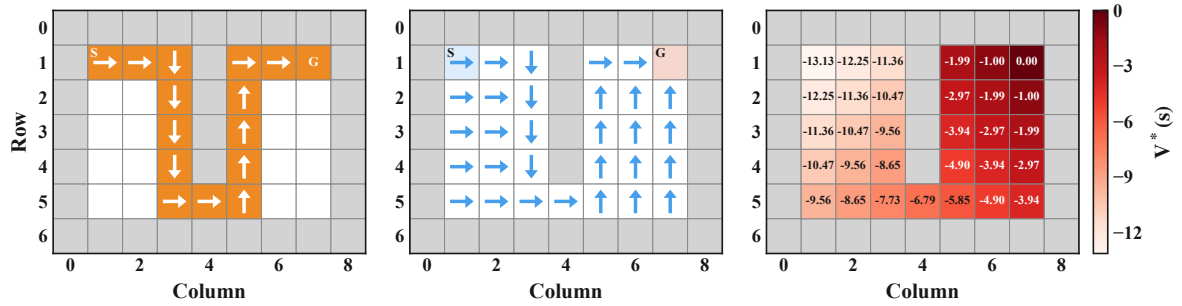
(2) 开放区域：`open_room`

去掉内部障碍后，起点到终点存在多条等长的最短路径，可以检查动作价值并列时的策略选择。两点的行列距离之和为 9，求得的路径也为 9 步，10 轮后停止。图中展示的是多个最优解中的一个，其具体走向由代码中的动作枚举顺序决定。



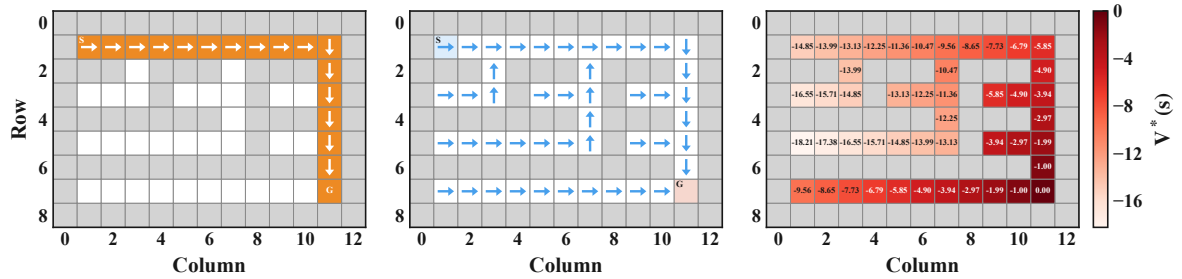
(3) 绕路障碍：detour

起点与终点位于同一行，但中间的墙只能从下方绕过，用来检查策略是否会为后续收益选择暂时远离终点的动作。两点的曼哈顿距离仅为 6，实际最短路径却需要 14 步。结果正确地先向下绕过墙体，再向上到达终点，15 轮后价值稳定。



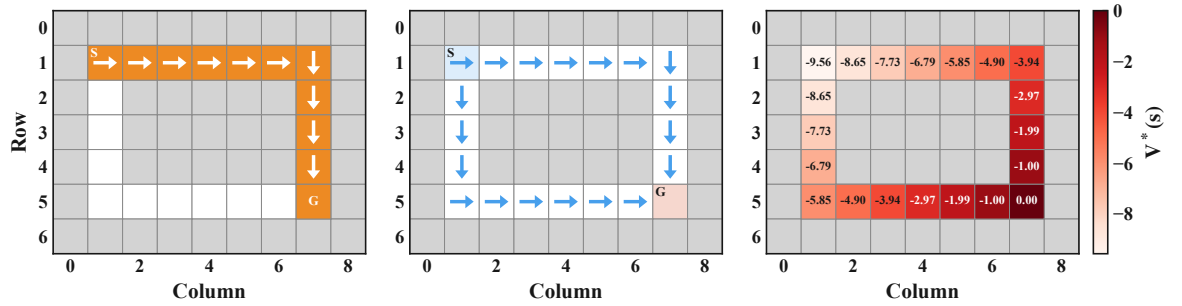
(4) 多条死胡同：dead_ends

这个测例加入了走入后必须返回的支路，用来检查价值是否能反映无效绕行的代价。最终从起点出发的策略沿 16 步最短路径到达终点；死胡同内部的箭头则指向出口，没有把智能体困在支路中。由于最远状态到终点仍需 20 步，整张价值表经过 21 轮才稳定。



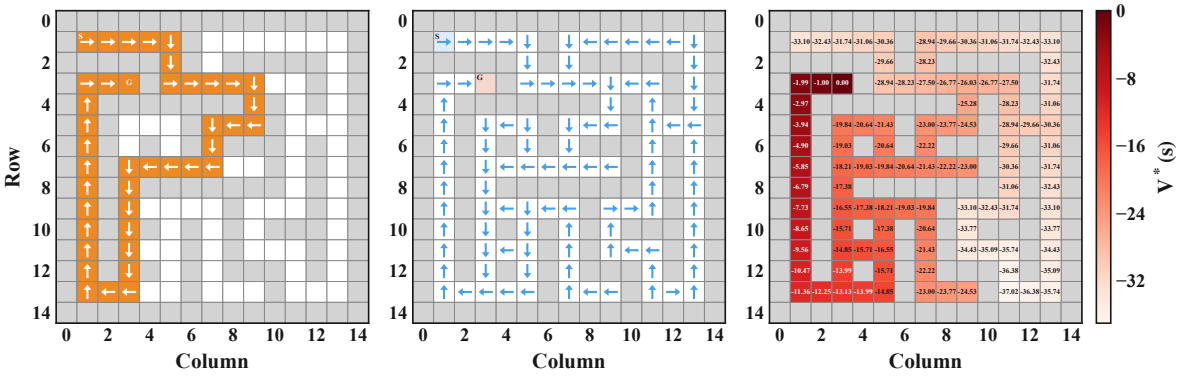
(5) 环形通道：loop

起点与终点之间有两条等长的绕行路线，用来检查环境存在回路时，最终策略是否仍会朝终点推进。算法选择了其中一条长度为 10 的最短路线，11 轮后停止。虽然迷宫结构有环，沿得到的策略执行时并没有出现循环。



(6) 组合结构：`larger`

最后将岔路、回路和狭窄通道放入 15×15 的迷宫，检查多种结构同时出现时的结果。起终点的曼哈顿距离只有 4，但墙壁使最短可行路径达到 40 步。算法求得了这条最短路径，起点价值为 -33.102824；由于部分支路距离终点更远，全局价值经过 47 轮才稳定。



5 总结

通过本次实验，我进一步理解了 MDP 的建模过程，以及 Value Iteration 如何通过迭代更新状态价值求得策略。将数学公式落实到代码，并结合不同测例观察结果，使我对状态、动作、奖励和价值函数之间的关系有了更直观的认识，也为后续学习强化学习算法打下了基础。